Bayesian Classifiers

Using probabilities and error costs to choose a class

Kangning Cui
City University of Hong Kong (Dongguan)

Semester A, 2026/27

Learning objectives

By the end of this lecture, you should be able to:

1. turn posterior probabilities into decisions using error costs;
2. derive Gaussian classifiers and explain LDA versus QDA;
3. build Naive Bayes classifiers for different feature types;
4. distinguish generative and discriminative classification;
5. choose evaluation measures for rare classes and unequal costs.

Probability estimation and decision making are related, but they are not the same problem.

Bayesian decision theory

CS5487 ▪ Semester A, 2026/27

Classification decisions

Suppose a model estimates:

$$p(y = 1 \mid \boldsymbol{x}) = 0.40, \qquad p(y = 0 \mid \boldsymbol{x}) = 0.60.$$

If both mistakes are equally serious, predicting class 0 seems natural.
But consider medical screening:

- false positive: an unnecessary follow-up test;
- false negative: a serious disease is missed.

The most probable class is not always the best decision.

Posterior vs. action

After observing $\boldsymbol{x}$, the posterior:

$$p(y = k \mid \boldsymbol{x})$$

describes uncertainty about the true class.
A decision rule must answer a different question:

What should we do given that uncertainty?

The answer can depend on:

- posterior probabilities;
- consequences of different mistakes.

Loss matrix

Let $\lambda(a, y)$ be the cost of choosing action a when the true class is y.
For binary classification:

	$y = 0$	$y = 1$
decide 0	0	C_{FN}
decide 1	C_{FP}	0

Here:

- C_{FP}: cost of a false positive;
- C_{FN}: cost of a false negative.

Different applications can assign very different costs.

Conditional risk

After observing $\boldsymbol{x}$, the true class is still uncertain.
For action a, average its possible costs using the posterior probabilities:

$$\boxed{\mathcal{R}(a \mid \boldsymbol{x}) = \sum_{y \in \mathcal{Y}} \lambda(a, y) p(y \mid \boldsymbol{x})}.$$

Here $\mathcal{Y}$ is the set of class labels, such as $\{0, 1\}$ for binary classification.
This is the **conditional risk**.
The Bayes decision chooses the action with the smallest expected cost:

$$\boxed{a^*(\boldsymbol{x}) = \arg\min_a \mathcal{R}(a \mid \boldsymbol{x})}.$$

A risk calculation

Suppose:

$$p(y = 1 \mid \boldsymbol{x}) = 0.35, \qquad p(y = 0 \mid \boldsymbol{x}) = 0.65,$$

with:

$$C_{\mathrm{FN}} = 5, \qquad C_{\mathrm{FP}} = 1.$$

If we decide class 0:

$$\mathcal{R}(0 \mid \boldsymbol{x}) = 5(0.35) = 1.75.$$

If we decide class 1:

$$\mathcal{R}(1 \mid \boldsymbol{x}) = 1(0.65) = 0.65.$$

Therefore:

$$\boxed{\text{decide class } 1}$$

even though class 1 has the smaller posterior probability.

Zero–one loss

Suppose every mistake has cost 1:

$$\lambda(a, y) = \mathbb{I}[a \neq y].$$

For action a:

$$\mathcal{R}(a \mid \boldsymbol{x}) = \sum_{y \neq a} p(y \mid \boldsymbol{x}).$$

Since posterior probabilities sum to one:

$$\mathcal{R}(a \mid \boldsymbol{x}) = 1 - p(a \mid \boldsymbol{x}).$$

Therefore:

$$\boxed{a^*(\boldsymbol{x}) = \arg\max_y p(y \mid \boldsymbol{x})}.$$

Maximum posterior classification is a special case of minimum-risk decision making.

Cost-sensitive threshold

For binary classification:

$$\mathcal{R}(0 \mid \boldsymbol{x}) = C_{\mathrm{FN}} p(y = 1 \mid \boldsymbol{x}),$$

while:

$$\mathcal{R}(1 \mid \boldsymbol{x}) = C_{\mathrm{FP}} p(y = 0 \mid \boldsymbol{x}).$$

Choose class 1 when:

$$C_{\mathrm{FP}} p(y = 0 \mid \boldsymbol{x}) < C_{\mathrm{FN}} p(y = 1 \mid \boldsymbol{x}).$$

If $p_1 = p(y = 1 \mid \boldsymbol{x})$, then:

$$\boxed{p_1 > \frac{C_{\mathrm{FP}}}{C_{\mathrm{FP}} + C_{\mathrm{FN}}}}.$$

The optimal threshold does not have to be 0.5.

Threshold example

Suppose:

$$C_{\mathrm{FP}} = 1, \qquad C_{\mathrm{FN}} = 5.$$

The optimal posterior threshold is:

$$\frac{1}{1+5} = \frac{1}{6} \approx 0.167.$$

So:

$$p(y = 1 \mid \boldsymbol{x}) > 0.167$$

is enough to predict class 1.
A high false-negative cost lowers the decision threshold.

Changing the cost can change the decision without retraining the probability model.

Bayes rule

How do we obtain the posterior?
Bayes' rule gives:

$$p(y = k \mid \boldsymbol{x}) = \frac{p(\boldsymbol{x} \mid y = k)p(y = k)}{p(\boldsymbol{x})}.$$

The two class-specific terms have different roles:

- $p(\boldsymbol{x} \mid y = k)$: how well class k explains the observed features;
- $p(y = k)$: how common class k is before seeing the features.

Likelihood and prior

Write the prior as:

$$\pi_k = p(y = k).$$

Under equal misclassification costs, the denominator $p(\boldsymbol{x})$ is shared by all classes. Therefore:

$$a^*(\boldsymbol{x}) = \arg\max_k p(\boldsymbol{x} \mid y = k)\pi_k.$$

In log space:

$$\boxed{a^*(\boldsymbol{x}) = \arg\max_k \left[\log p(\boldsymbol{x} \mid y = k) + \log \pi_k\right]}.$$

Classification combines feature evidence and class frequency.

Likelihood ratio

For two classes, Bayes' rule gives:

$$\frac{p(y=1\,|\,\boldsymbol{x})}{p(y=0\,|\,\boldsymbol{x})} = \frac{p(\boldsymbol{x}\,|\,y=1)}{p(\boldsymbol{x}\,|\,y=0)}\frac{\pi_1}{\pi_0}.$$

The three quantities are:

- **Posterior odds:** the ratio on the left;
- **Likelihood ratio:** the first ratio on the right;
- **Prior odds:** π_1/π_0.

With unequal error costs, predict class 1 when:

$$\boxed{\frac{p(\boldsymbol{x}\,|\,y=1)}{p(\boldsymbol{x}\,|\,y=0)} > \frac{C_{\mathrm{FP}}}{C_{\mathrm{FN}}}\frac{\pi_0}{\pi_1}}.$$

The required evidence depends on both class frequency and error cost.

Priors and costs

Priors and costs play different roles.

- **Prior:** how often a class occurs.
- **Cost:** how serious a wrong decision is.

A rare class can still have a very high false-negative cost.
For example:

$$\text{rare disease} \neq \text{unimportant disease.}$$

Do not change a prior just because a class is important.

Decision regions

A classifier divides feature space into decision regions.
A decision region contains all feature values assigned to the same class.
For class k:

$$\boxed{\mathcal{R}_k = \{\boldsymbol{x} : a^*(\boldsymbol{x}) = k\}}.$$

For two classes, the boundary occurs where neither decision has lower risk:

$$\mathcal{R}(0 \,|\, \boldsymbol{x}) = \mathcal{R}(1 \,|\, \boldsymbol{x}).$$

Under equal error costs, this means:

$$p(y = 0 \,|\, \boldsymbol{x}) = p(y = 1 \,|\, \boldsymbol{x}).$$

A Gaussian model determines where this equality occurs.

A simple example

Consider two classes with equal priors and equal error costs.
Suppose:

$$x \mid y = 0 \sim \mathcal{N}(0, 1), \qquad x \mid y = 1 \sim \mathcal{N}(2, 1).$$

Under equal costs, the boundary occurs when the two posterior probabilities are equal.
Because the priors are also equal, this is equivalent to:

$$p(x \mid y = 0) = p(x \mid y = 1).$$

For these two Gaussian distributions:

$$e^{-x^2/2} = e^{-(x-2)^2/2}.$$

Therefore:

$$x^2 = (x-2)^2 \quad \Longrightarrow \quad \boxed{x = 1}.$$

Decision regions

The point: $x = 1$ is the decision boundary.
For:

$$x < 1,$$

the observation is closer to the class-0 distribution, so:

$$\boxed{\mathcal{R}_0 = \{x : x < 1\}}.$$

For:

$$x > 1,$$

the observation is closer to the class-1 distribution, so:

$$\boxed{\mathcal{R}_1 = \{x : x > 1\}}.$$

Thus:

$$\boxed{\text{probability model} \rightarrow \text{equal-risk boundary} \rightarrow \text{decision regions}}.$$

Gaussian classifiers generalize this idea to multiple features and more flexible class shapes.

Gaussian classifiers

CS5487 ▪ Semester A, 2026/27

Gaussian class model

Suppose each class forms a cloud in feature space.
Model class k as:

$$\boldsymbol{x} \mid y = k \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

Three quantities describe the class:

- $\boldsymbol{\mu}_k$: the center of the cloud;
- $\boldsymbol{\Sigma}_k$: its spread and orientation;
- π_k: how common the class is.

Classification asks which class distribution best explains a new observation.

Gaussian class score

Under equal error costs, compare:

$$p(\boldsymbol{x} \mid y = k)\pi_k.$$

Taking logarithms gives the Gaussian score:

$$\boxed{g_k(\boldsymbol{x}) = -\frac{1}{2} d_k^2(\boldsymbol{x}) - \frac{1}{2} \log |\boldsymbol{\Sigma}_k| + \log \pi_k}$$

where:

$$d_k^2(\boldsymbol{x}) = (\boldsymbol{x} - \boldsymbol{\mu}_k)^\mathsf{T} \boldsymbol{\Sigma}_k^{-1} (\boldsymbol{x} - \boldsymbol{\mu}_k).$$

The score combines distance, spread, and prior probability.
Predict:

$$\boxed{\hat{y} = \arg\max_k g_k(\boldsymbol{x})}.$$

Covariance and distance

The Gaussian distance is called the squared Mahalanobis distance:

$$d_k^2(\boldsymbol{x}) = (\boldsymbol{x} - \boldsymbol{\mu}_k)^\mathsf{T} \boldsymbol{\Sigma}_k^{-1} (\boldsymbol{x} - \boldsymbol{\mu}_k).$$

Unlike ordinary Euclidean distance, it accounts for the class's natural variation.
Consider:

$$\boldsymbol{\Sigma}_A = \boldsymbol{I}, \qquad \boldsymbol{\Sigma}_B = \begin{bmatrix} 1 & 0 \\ 0 & 4 \end{bmatrix}.$$

For a displacement $(0, 2)^\mathsf{T}$ from the mean:

$$d_A^2 = 4, \qquad d_B^2 = 1.$$

The same displacement is less unusual for class B because its second feature varies more.

Prior effect

Consider the earlier one-dimensional Gaussian example:

$$x \mid y = 0 \sim \mathcal{N}(0, 1), \qquad x \mid y = 1 \sim \mathcal{N}(2, 1).$$

At $x = 1.2$, the likelihoods are approximately:

$$p(x \mid 0) = 0.19, \qquad p(x \mid 1) = 0.29.$$

Priors	Class 0 score	Class 1 score	Predict
Equal $(0.5, 0.5)$	0.095	0.145	1
Unequal $(0.8, 0.2)$	0.152	0.058	0

The scores are likelihood multiplied by prior.

A less common class needs stronger feature evidence to be predicted.

Decision boundary

For two classes, predict class 1 when:

$$g_1(\boldsymbol{x}) > g_0(\boldsymbol{x}).$$

The boundary occurs where their scores are equal:

$$\boxed{g_1(\boldsymbol{x}) = g_0(\boldsymbol{x})}.$$

Its shape depends on the Gaussian covariance assumptions.

- **QDA:** classes can have different covariance matrices.
- **LDA:** classes share one covariance matrix.

We first consider what happens when class shapes differ.

QDA

Quadratic discriminant analysis (QDA) allows each class to have its own covariance:

$$\boldsymbol{\Sigma}_0, \boldsymbol{\Sigma}_1, \ldots, \boldsymbol{\Sigma}_{K-1}.$$

Different covariance matrices allow different:

- spreads along each feature;
- orientations and correlations;
- distance penalties.

As a result, the class scores can change differently across feature space.
The decision boundary can therefore be curved.

QDA example

Consider two classes with the same center and equal priors:

$$\boldsymbol{\mu}_0 = \boldsymbol{\mu}_1 = \mathbf{0}, \qquad \pi_0 = \pi_1 = 0.5.$$

Let:

$$\boldsymbol{\Sigma}_0 = \boldsymbol{I}, \qquad \boldsymbol{\Sigma}_1 = 4\boldsymbol{I}.$$

Class 0 is concentrated near the origin.
Class 1 is more spread out.
Therefore:

- near the center, class 0 has higher density;
- far from the center, class 1 decreases more slowly.

The preferred class changes as we move away from the center.

QDA scores

Let $r^2 = x_1^2 + x_2^2$ measure squared distance from the origin.
For the previous example, ignoring the common prior term:

$$g_0(\boldsymbol{x}) = -\frac{1}{2}r^2, \qquad g_1(\boldsymbol{x}) = -\log 4 - \frac{1}{8}r^2.$$

Compare two locations:

- At $r = 0$: $g_0 = 0$, $g_1 \approx -1.39$. Predict class 0.
- At $r = 3$: $g_0 = -4.50$, $g_1 \approx -2.51$. Predict class 1.

The scores become equal at approximately $r = 1.92$.
Thus the boundary is a circle:

$$\boxed{x_1^2 + x_2^2 \approx 3.70}.$$

Why quadratic?

The previous boundary contains:

$$x_1^2 + x_2^2.$$

These squared terms remain because the two classes use different covariance matrices.
More generally, the boundary equation can contain:

$$x_1^2, \qquad x_2^2, \qquad x_1 x_2.$$

Such terms can describe curved boundaries.
This explains the name:

Quadratic Discriminant Analysis.

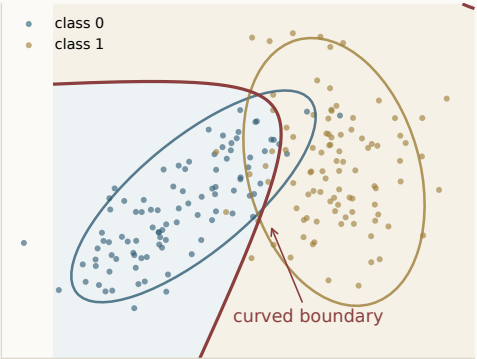
The exact shape depends on the class means, covariances, and priors.

QDA boundary

Our simple example produced a circle.
With more general covariance matrices, QDA can produce other quadratic boundaries.
The boundary is always determined by:

$$g_0(\boldsymbol{x}) = g_1(\boldsymbol{x}).$$

QDA is flexible because each class can have its own shape.
However, it must estimate more parameters.



Shared covariance

Now suppose all classes share the same covariance:

$$\boxed{\boldsymbol{\Sigma}_k = \boldsymbol{\Sigma}}.$$

The classes can have different centers, but their covariance structure is identical.
This means that all class scores contain the same quadratic term.
When comparing the scores, that common term disappears.
The resulting decision boundary is linear.
This is the idea behind linear discriminant analysis (LDA).

LDA example

Consider two classes with equal priors:

$$\boldsymbol{\mu}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \qquad \boldsymbol{\mu}_1 = \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \qquad \boldsymbol{\Sigma} = \boldsymbol{I}.$$

The classes have identical spread.
The boundary occurs where the squared distances are equal:

$$x_1^2 + x_2^2 = (x_1 - 2)^2 + x_2^2.$$

Expanding the right-hand side:

$$x_1^2 + x_2^2 = x_1^2 - 4x_1 + 4 + x_2^2.$$

The squared terms cancel, leaving:

$$\boxed{x_1 = 1}.$$

This is a straight line.

Why linear?

The previous example shows what cancellation means.
With shared covariance, both Gaussian scores contain the same quadratic part:

$$-\frac{1}{2}\boldsymbol{x}^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{x}.$$

When one score is subtracted from the other, this term disappears.
The remaining difference has the form:

$$\boxed{g_1(\boldsymbol{x}) - g_0(\boldsymbol{x}) = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + b}.$$

Therefore the boundary:

$$\boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + b = 0$$

is linear.

Binary LDA

For two classes with shared covariance, the weight vector is:

$$\boxed{\boldsymbol{w} = \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)}.$$

Two quantities determine this direction:

- the difference between class centers;
- the natural variation within the classes.

The classifier predicts class 1 when:

$$\boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + b > 0.$$

Thus LDA produces an ordinary linear decision rule.

LDA weights

Consider:

$$\boldsymbol{\mu}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \qquad \boldsymbol{\mu}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix},$$

with shared covariance:

$$\boldsymbol{\Sigma} = \begin{bmatrix} 1 & 0 \\ 0 & 4 \end{bmatrix}.$$

The LDA weight vector is:

$$\boldsymbol{w} = \begin{bmatrix} 1 & 0 \\ 0 & 1/4 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \boxed{\begin{bmatrix} 2 \\ 0.25 \end{bmatrix}}.$$

The second feature receives less weight because it has larger within-class variance.

LDA boundary

For the previous example, assume equal priors.
The intercept is:

$$b = -\frac{1}{2}(\boldsymbol{\mu}_1 + \boldsymbol{\mu}_0)^\mathsf{T}\boldsymbol{w} = -\frac{1}{2}(4 + 0.25) = \boxed{-2.125}.$$

Therefore the classifier is:

$$\boxed{f(\boldsymbol{x}) = 2x_1 + 0.25x_2 - 2.125}.$$

Its decision boundary is:

$$\boxed{2x_1 + 0.25x_2 - 2.125 = 0}.$$

For $\boldsymbol{x}_A = (1, 1)^\mathsf{T}$, the score is $0.125 > 0$, so predict class 1.
For $\boldsymbol{x}_B = (0.5, 2)^\mathsf{T}$, the score is $-0.625 < 0$, so predict class 0.

Parameter estimation

Both LDA and QDA estimate class priors and means from labeled data.
For class k with n_k observations:

$$\hat{\pi}_k = \frac{n_k}{n}, \qquad \hat{\boldsymbol{\mu}}_k = \frac{1}{n_k} \sum_{i:y_i=k} \boldsymbol{x}_i.$$

They differ in covariance estimation:

- **QDA:** estimate one covariance matrix per class.
- **LDA:** pool within-class variation to estimate one shared covariance matrix.

Pooling reduces the number of parameters that must be estimated.

LDA or QDA?

	LDA	QDA
Covariance	shared	class-specific
Boundary	linear	generally quadratic
Flexibility	lower	higher
Parameters	fewer	more
Data need	lower	higher

LDA is attractive when covariance structures are similar or training data are limited.
QDA can help when class covariance structures genuinely differ and enough data are available.

Covariance limits

A full covariance matrix requires many parameters as the number of features grows.
With limited training data:

- covariance estimates become noisy;
- matrix inversion can become unstable;
- QDA may overfit.

Sharing covariance makes LDA simpler and often more stable.
A further simplification is to model features separately.
This leads to Naive Bayes.

Naive Bayes

CS5487 ▪ Semester A, 2026/27

Density challenge

Suppose $\boldsymbol{x}$ has hundreds or thousands of features.
Directly estimating:

$$p(\boldsymbol{x} \mid y = k)$$

can require modeling many dependencies among those features.
With limited data, this becomes difficult.
Naive Bayes asks:

What if we model each feature separately?

This greatly reduces the estimation problem.

Naive assumption

Naive Bayes assumes that features are conditionally independent once the class is known:

$$p(\boldsymbol{x} \mid y = k) = \prod_{j=1}^{d} p(x_j \mid y = k).$$

For example, suppose two features indicate whether the words “free” and “offer” appear in an email.
Once we know whether the email is spam, Naive Bayes treats the two word occurrences as independent.
This does **not** mean the features are independent across the entire dataset.
The assumption is strong, but it makes probability estimation much simpler.

Log scores

Using Bayes classification:

$$p(y = k \mid \boldsymbol{x}) \propto \pi_k \prod_{j=1}^{d} p(x_j \mid y = k).$$

Taking logs gives:

$$\boxed{g_k(\boldsymbol{x}) = \log \pi_k + \sum_{j=1}^{d} \log p(x_j \mid y = k)}.$$

Log space provides two benefits:

- products become sums;
- numerical underflow is reduced.

Bernoulli model

For binary features:

$$x_j \in \{0, 1\}.$$

Let:

$$\theta_{jk} = p(x_j = 1 \mid y = k).$$

Then:

$$p(\boldsymbol{x} \mid y = k) = \prod_j \theta_{jk}^{x_j} (1 - \theta_{jk})^{1 - x_j}.$$

This is useful when each feature represents presence or absence.
Examples include binary indicators and word occurrence.

Zero probabilities

Suppose a feature never appears in the training examples of one class.
The maximum-likelihood estimate may give:

$$\hat{\theta}_{jk} = 0.$$

Then any test example containing that feature gets:

$$p(\boldsymbol{x} \mid y = k) = 0.$$

One unseen event can therefore eliminate an entire class.
We need smoothing.

Smoothing

For a binary feature, add a pseudo-count $\alpha > 0$:

$$\boxed{\hat{\theta}_{jk} = \frac{N_{jk} + \alpha}{N_k + 2\alpha}}.$$

Here:

- N_{jk}: class-k samples with feature $j = 1$;
- N_k: total number of class-k samples;
- α: smoothing strength.

The denominator adds 2α because a binary feature has two possible outcomes: 0 and 1. With $\alpha = 1$, even an unseen feature no longer receives probability zero.

Smoothing example

Suppose class k contains four examples.
One feature appears in all four:

$$N_{jk} = 4.$$

With $\alpha = 1$:

$$\hat{\theta}_{jk} = \frac{4+1}{4+2} = \frac{5}{6}.$$

If another feature is never observed:

$$\hat{\theta} = \frac{0+1}{4+2} = \frac{1}{6}.$$

Smoothing avoids absolute certainty from a small dataset.

Bernoulli example

Suppose:

$$\pi_1 = \pi_0 = 0.5.$$

For class 1:

$$p(x_1 = 1 \mid 1) = 0.8, \qquad p(x_2 = 1 \mid 1) = 0.6.$$

For class 0:

$$p(x_1 = 1 \mid 0) = 0.2, \qquad p(x_2 = 1 \mid 0) = 0.4.$$

For $\boldsymbol{x} = (1, 1)$:

$$\text{score}_1 = 0.5(0.8)(0.6) = 0.24,$$

$$\text{score}_0 = 0.5(0.2)(0.4) = 0.04.$$

Predict class 1.

Posterior normalization

The previous values are unnormalized class scores:

$$s_1 = 0.24, \qquad s_0 = 0.04.$$

Normalize them:

$$p(y = 1 \mid \boldsymbol{x}) = \frac{0.24}{0.24 + 0.04}.$$

Therefore:

$$\boxed{p(y = 1 \mid \boldsymbol{x}) \approx 0.857}.$$

For classification, normalization is unnecessary if we only need the largest score.
For probability estimates, normalization is required.

Multinomial model

Multinomial Naive Bayes is useful for word-count data.
Let x_j be the number of occurrences of word j:

$$x_j = 0, 1, 2, \ldots$$

Here θ_{jk} is the probability of word j under class k, with $\sum_j \theta_{jk} = 1$.
The likelihood is proportional to:

$$p(\boldsymbol{x} \mid y = k) \propto \prod_j \theta_{jk}^{x_j}.$$

Taking logarithms gives the class score:

$$\boxed{g_k(\boldsymbol{x}) = \log \pi_k + \sum_j x_j \log \theta_{jk}}.$$

Unlike Bernoulli Naive Bayes, repeated word occurrences contribute repeatedly to the score.

Gaussian Naive Bayes

For continuous features, model each feature separately:

$$x_j \mid y = k \sim \mathcal{N}(\mu_{jk}, \sigma_{jk}^2).$$

Then:

$$p(\boldsymbol{x} \mid y = k) = \prod_j \mathcal{N}(x_j \mid \mu_{jk}, \sigma_{jk}^2).$$

This is equivalent to using a diagonal covariance matrix for each class.
No cross-feature covariance terms are estimated.

Why it can work

Real features are often not conditionally independent.
Why can Naive Bayes still classify well?

- It estimates far fewer parameters.
- Individual feature evidence can still point toward the correct class.
- Classification only requires the correct ordering of class scores.

The probability values themselves can still be inaccurate.

Overconfident probabilities

If two correlated features contain nearly the same information, Naive Bayes may count that evidence twice.
This can make:

$$p(y = k \mid \boldsymbol{x})$$

too close to 0 or 1.
Therefore:

- good accuracy does not imply good probability estimates;
- probability quality should be evaluated separately.

This distinction becomes important when decisions depend on risk.

When to use it

Naive Bayes is especially useful when:

- features are high-dimensional;
- data are sparse;
- training data are limited;
- a fast interpretable baseline is needed.

Typical examples include text classification and simple diagnostic models.
Its main strength is:

simple estimation from limited data.

Generative versus discriminative

CS5487 ▪ Semester A, 2026/27

Two routes

Generative

Model:

$$p(\boldsymbol{x} \mid y) \quad \text{and} \quad p(y).$$

Then obtain:

$$p(y \mid \boldsymbol{x})$$

using Bayes' rule.
Examples:

- LDA and QDA;
- Naive Bayes.

Discriminative

Model directly:

$$p(y \mid \boldsymbol{x})$$

or a decision score.
Examples include:

- logistic regression;
- support vector machines;
- neural classifiers.

Different questions

A generative model asks:

How could each class generate the observed features?

A discriminative model asks:

How should the classes be separated?

Generative models describe more of the data distribution.
Discriminative models can focus directly on prediction.
Neither approach is universally better.

Assumptions and data

Generative models often make stronger distributional assumptions.
This can help when:

- training data are limited;
- the assumptions are approximately correct;
- class densities are themselves useful.

Discriminative models often become attractive when:

- enough labeled data are available;
- the boundary is more important than the full feature distribution.

The comparison is another bias–variance trade-off.

Classifier evaluation

CS5487 ▪ Semester A, 2026/27

Accuracy can mislead

Suppose only 1% of examples are positive.
A classifier predicts every example as negative.
Its accuracy is:

$$\boxed{99\%}.$$

But its sensitivity is:

$$\boxed{0\%}.$$

The classifier never finds the positive class.
A useful metric must reflect the goal of the application.

Confusion matrix

Consider:

	Actual positive	Actual negative
Predicted positive	$TP = 18$	$FP = 12$
Predicted negative	$FN = 2$	$TN = 68$

The four counts describe different outcomes:

- TP: positive case found;
- FN: positive case missed;
- FP: false alarm;
- TN: negative case correctly rejected.

Core metrics

Sensitivity (Recall or TPR) measures how many actual positives are found:

$$\text{Sensitivity} = \frac{TP}{TP + FN}.$$

Specificity (TNR) measures how many actual negatives are correctly rejected:

$$\text{Specificity} = \frac{TN}{TN + FP}.$$

Precision measures how many predicted positives are correct:

$$\text{Precision} = \frac{TP}{TP + FP}.$$

The false-positive rate is $FPR = 1 - \text{Specificity}$.

Metric calculation

Using:

$$TP = 18, \quad FN = 2, \quad FP = 12, \quad TN = 68,$$

we obtain:

$$\text{Sensitivity} = \frac{18}{20} = 0.90,$$

$$\text{Specificity} = \frac{68}{80} = 0.85,$$

and:

$$\text{Precision} = \frac{18}{30} = 0.60.$$

The same classifier can score very differently under different metrics.

Threshold trade-off

Suppose class 1 is predicted when:

$$p(y = 1 \mid \boldsymbol{x}) > \tau.$$

Lowering τ usually gives:

more positive predictions

$\Downarrow$

higher sensitivity and more false positives.

Raising τ produces the opposite trade-off.
There is no universally best threshold.

ROC curve

Change the threshold τ and record:

$$\text{TPR} = \frac{TP}{TP + FN},$$

and:

$$\text{FPR} = \frac{FP}{FP + TN}.$$

The ROC curve plots:

$$\boxed{\text{TPR versus FPR}}$$

across thresholds.
It measures ranking behavior rather than performance at one fixed threshold.

Precision–recall curve

For rare positive classes, false positives can dominate practical usefulness.
The precision–recall curve compares:

$$\text{Precision} = \frac{TP}{TP + FP}$$

with:

$$\text{Recall} = \frac{TP}{TP + FN}.$$

It emphasizes performance on the positive class.
This is often informative when positive events are rare.

Calibration

A classifier may rank examples correctly but produce poor probabilities.
Calibration asks:

Do predicted probabilities match observed frequencies?

For example, among many cases predicted near:

$$p(y = 1 \mid \boldsymbol{x}) = 0.8,$$

approximately 80% should actually be positive.
Calibration matters when probabilities are used for downstream decisions.

Decision cost

If false-negative and false-positive costs are known, evaluate the decisions directly.
For a test set of size N:

$$\boxed{\text{Average cost} = \frac{C_{\text{FN}} FN + C_{\text{FP}} FP}{N}}.$$

For:

$$FN = 2, \quad FP = 12,$$

with:

$$C_{\text{FN}} = 10, \qquad C_{\text{FP}} = 1,$$

the average cost is:

$$\frac{20 + 12}{100} = 0.32.$$

Prior shift

Suppose the frequency of positive cases changes after deployment:

$$\pi_1^{\text{train}} \neq \pi_1^{\text{deploy}}.$$

If the class-conditional distributions $p(\boldsymbol{x} \mid y)$ remain unchanged, we have **prior shift** (or label shift).
In this case:

- the class-conditional models can remain unchanged;
- posterior probabilities must account for the new class priors;
- deployment priors must be estimated reliably.

This differs from a change in $p(\boldsymbol{x} \mid y)$, where the feature distribution within a class has changed.

Evaluation checklist

Before reporting one number, ask:

- Are the classes balanced?
- Which mistake matters most?
- Is ranking important?
- Are probability values themselves used?
- Will the deployment class frequency differ?

Different questions require different evaluation measures.

Accuracy, ranking, probability quality, and decision cost are different properties.

Week 7 summary

The main ideas are:

1. Posterior probabilities describe uncertainty; decision costs determine actions.
2. Bayes decisions minimize posterior expected loss.
3. Gaussian class models lead to QDA or LDA depending on covariance assumptions.
4. Naive Bayes simplifies high-dimensional density estimation through conditional independence.
5. Classification accuracy, probability quality, and decision cost should be evaluated separately.

$$\boxed{\text{probability model} \rightarrow \text{posterior} \rightarrow \text{decision rule} \rightarrow \text{evaluation}}$$

Probabilities become decisions only after costs are specified

Questions and discussion